

空调智能客服系统 - RAG 项目说明文档

一、项目基本信息

项目名称

空调智能客服系统 (Air Conditioner Intelligent Customer Service System)

应用场景

为空调用户提供实时、准确的产品使用咨询、故障排查、选购建议、维护保养等多方面的智能服务。通过 RAG 技术实现从企业知识库中检索相关内容，结合大语言模型进行专业、流畅的自然语言回答。

核心功能

- ☒ **选购指导**: 基于用户需求和环境条件推荐合适的空调产品
- ☒ **使用说明**: 详细解答空调操作、功能使用等问题
- ☒ **故障排除**: 诊断常见故障并提供解决方案
- ☒ **维护保养**: 指导用户进行日常维护和清洁
- ☒ **能耗常识**: 提供节能建议和能耗相关知识

二、目标用户与使用场景

目标用户

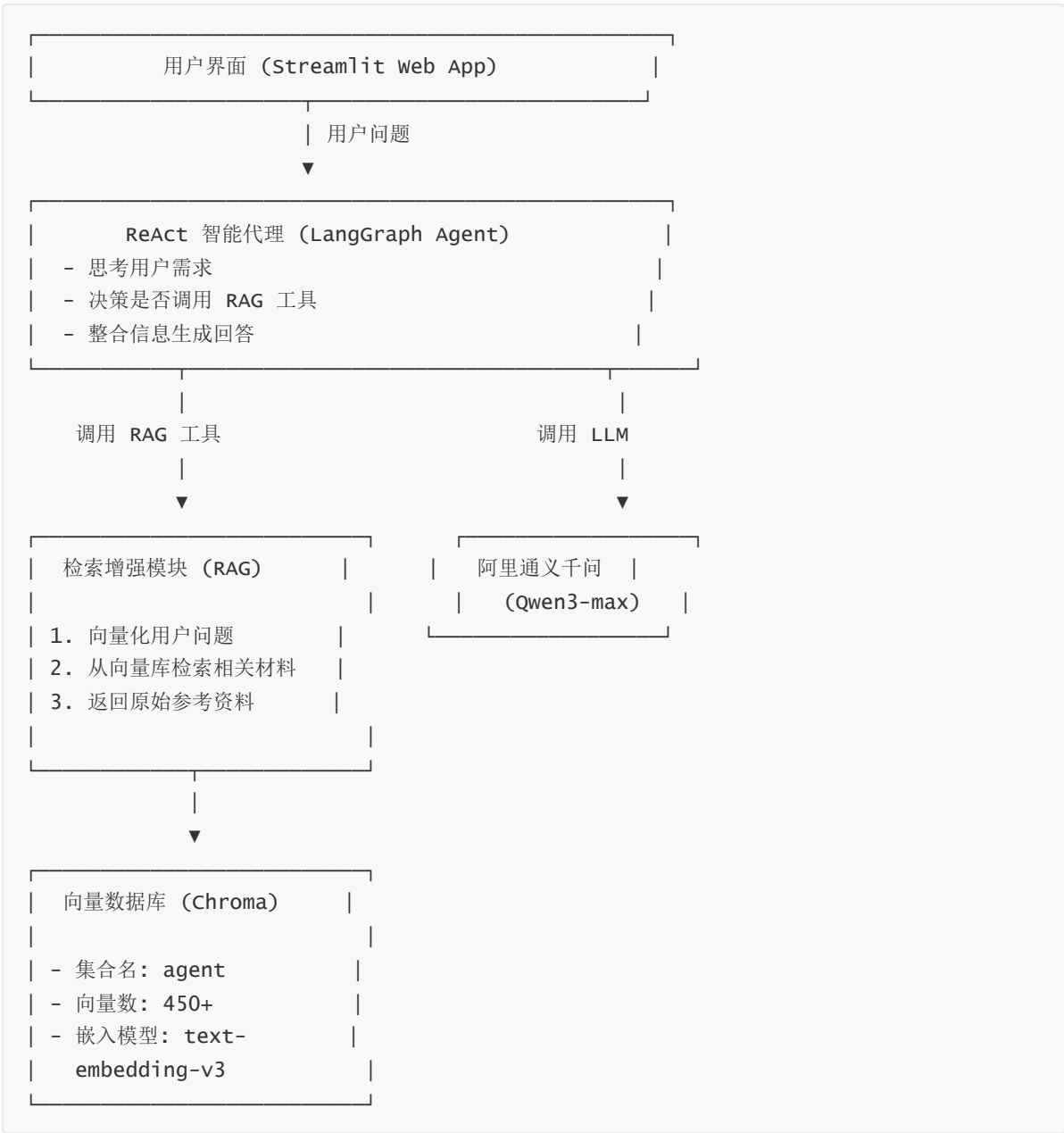
-  空调消费者 (咨询选购、使用、维修)
-  维修人员 (快速查阅故障排查方案)
-  客服人员 (辅助支持复杂问题回答)
-  企业知识库管理者 (维护和更新知识库)

使用场景

场景	问题示例	系统回答
选购咨询	"小户型适合哪些空调？"	基于空调知识库推荐适配产品
故障排查	"空调不制冷怎么办？"	返回诊断步骤和解决方案
使用问题	"空调怎么调节温度？"	提供详细操作说明
维护建议	"空调怎么清洁保养？"	提供保养步骤和注意事项

三、RAG 系统架构设计

整体架构流程



核心组件说明

组件	技术	职责
智能代理	LangGraph ReAct	决策引擎，控制工具调用流程
LLM 模型	阿里通义千问 qwen3-max	语言理解与生成
向量化	DashScopeEmbeddings	将文本转换为向量
向量库	Chroma	存储和检索相似文档
前端	Streamlit	用户交互界面

四、数据处理流程

4.1 数据加载

数据来源：data/ 文件夹下的知识库文件

```
data/
├─ 空调选购指南.txt      (选购相关知识)
├─ 空调使用说明.txt      (操作说明)
├─ 空调故障排除.txt      (故障解决方案)
├─ 空调维护保养.txt      (保养知识)
└─ 空调能耗常识.txt      (节能知识)
```

加载方式：

```
from utils.file_handler import txt_loader
docs = txt_loader("data/空调故障排除.txt")
```

4.2 数据清洗

清洗步骤：

1. 去重处理

- 按内容前 100 字符进行去重
- 移除完全相同的文档

2. 内容验证

- 过滤长度 < 10 字符的文档
- 去除重复字符过多的文本

3. 格式标准化

- 去除多余空格和换行符
- 保留中文标点和英文字符

关键代码：

```
from data_loader_cleaner import DataCleaner

cleaner = DataCleaner()
docs = cleaner.remove_duplicates(docs)
docs = cleaner.filter_documents(docs)
```

4.3 文本分块

分块策略：使用递归字符分割器 (RecursiveCharacterTextSplitter)

参数配置：

```
chunk_size: 100          # 每块约 100 字符（中文约 33 字）
chunk_overlap: 20        # 块间重叠 20 字符，保证上下文连贯
separators:
  - "\n\n"              # 优先按段落分割
  - "\n"                # 其次按换行分割
  - "。"                # 再次按句号分割
  - "，"                # 按顿号分割
  - " "                 # 按空格分割
  - ""                  # 最后按字符分割
```

分块示例：

原始文本：
"故障现象：空调不制冷。检测方法：检查温度设置、清洁滤网。
修复方案：调整温度，清洁或更换滤网。"

↓ 分块后 ↓

块1: "故障现象：空调不制冷。检测方法：检查温度设置、"
块2: "查温度设置、清洁滤网。修复方案：调整温度，清洁或"
块3: "或更换滤网。"

处理结果：

5 个源文件 → 500+ 个文本块 → 450+ 个向量

五、检索方案说明

5.1 检索方式

相似度检索 (Similarity Search)

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
relevant_docs = retriever.invoke("空调不制冷")
```

工作流程：

1. 用户问题向量化
2. 计算与库中所有向量的余弦相似度
3. 返回相似度最高的 k 个文档块 (k=3)

5.2 检索优化

问题向量化：

```
from model.factory import embed_model

query_vector = embed_model.embed_query("空调怎么清洁? ")
# 输出: [0.123, -0.456, 0.789, ...] (向量维度: 1024)
```

相似度计算：

$$\begin{aligned}\text{相似度} &= (\text{query_vector} \cdot \text{doc_vector}) / (|\text{query_vector}| \times |\text{doc_vector}|) \\ &= \text{余弦相似度} \in [-1, 1]\end{aligned}$$

5.3 检索效果评估

指标	说明	目标值
召回率 (Recall)	相关文档被检索出的比例	> 80%
准确率 (Precision)	检索结果中相关的比例	> 75%
MRR (Mean Reciprocal Rank)	相关结果排名倒数的平均值	> 0.8

六、Prompt 设计

6.1 系统提示词 (System Prompt)

位置：prompts/main_prompt.txt

核心内容：

你是空调领域的专业智能客服，具备自主的ReAct思考与工具调用能力，严格遵循「思考→行动→观察→再思考」的流程回答用户问题，思考过程需贴合空调的应用场景，逻辑自洽。

核心思考准则

- 先判断用户的核心需求，分析当前已有的信息是否足够直接回答
- 若不足，思考需要调用什么工具获取缺失信息
- 工具返回结果后，再次判断信息是否完整
- 若信息足够，整合生成流畅、专业的最终回答
- 若5次工具调用后仍信息不足，则回复用户：我不知道

6.2 工具定义

RAG 检索工具：

工具名：rag_retrieve

描述：从向量库检索空调相关资料

入参：query（检索词）

出参：原始参考资料内容（纯文本，无格式标签）

约束条件：

- 工具仅返回原始资料，不进行总结
- LLM 需要自主思考、整合和重组
- 生成流畅专业的自然语言回答

6.3 Prompt 示例

示例1：选购咨询

用户问题：小户型适合哪些空调？

系统思考：

- 1. 用户需要了解小户型空调选购建议
- 2. 需要从知识库检索相关信息
- 3. 使用 RAG 工具查询"小户型空调选购"

系统行动：

调用 rag_retrieve("小户型空调选购")

系统观察：

返回参考资料（选购指南中的小户型相关内容）

最终回答：

基于参考资料，生成包含具体产品、功率、特点的专业回答

示例2：故障排除

用户问题：空调开了但不出风怎么办？

系统思考：

- 1. 这是空调故障问题
- 2. 需要获取诊断和解决方案
- 3. 使用 RAG 工具查询"空调不出风"

系统行动：

调用 rag_retrieve("空调故障 不出风")

系统观察：

返回故障排除方案（检测步骤、修复方法）

最终回答：

整合参考资料，提供清晰的诊断流程和解决步骤

七、工具与框架接入说明

7.1 技术栈

层级	技术	用途
应用层	Streamlit	Web UI
智能体层	LangGraph	ReAct 代理框架
LLM层	LangChain + Dashscope	LLM 调用
嵌入层	DashScopeEmbeddings	文本向量化
向量库	Chroma	向量存储与检索

层级	技术	用途
数据处理	LangChain TextSplitters	文本分块

7.2 关键代码结构

```
agent/react_agent.py
├─ class ReactAgent
│   ├── __init__()          # 初始化智能体
│   ├── execute_stream()    # 流式执行查询
│   └─ 使用 create_react_agent() 创建 ReAct 框架

agent/tools/agent_tools.py
├─ @tool def rag_retrieve() # RAG 检索工具
├─ @tool def get_weather()  # 天气工具（示例）
└─ ...（其他工具）

rag/rag_service.py
├─ class RagSummarizeService
│   ├── retrieve_raw_context() # 检索原始资料
│   └─ retriever_docs()       # 获取检索器

model/factory.py
├─ ChatModelFactory()        # LLM 工厂
└─ EmbeddingsFactory()       # 嵌入模型工厂
```

7.3 框架初始化流程

```
# 1. 创建 ReAct 智能体
from agent.react_agent import ReactAgent
agent = ReactAgent()

# 2. 执行查询
for chunk in agent.execute_stream("空调怎么选? "):
    print(chunk, end="", flush=True)

# 3. 返回流式结果
# "根据空调知识库，小户型空调选购建议如下：
# 1. 制冷量选择 ...
# 2. 能效等级 ...
# 3. 品牌推荐 ..."
```

八、示例输入输出

用户输入：

空调不制冷怎么办？

系统输出：



空调不制冷怎么办？



空调不制冷怎么办？ 空调完全不通电、无任何反应，首先检查电源插头是否松动脱落。

确认插座是否有电，可用手机充电器等其他电器插入测试。

检查配电箱内负责空调回路的空气开关是否跳闸，若跳闸尝试复位。新安装的空调制冷差，可能是抽真空不彻底导致系统内混有空气，严重影响冷效。

空调使用三五年后制冷下降，不一定是缺氟，不要被无良维修工忽悠，先清洗内外机。遥控器控制距离变短，必须贴近内机才能操作，更换新电池仍无效则是内机接收头受潮老化。

当室外气温超过43度，大部分空调会进行高压保护降频运转，制冷效果变差，并非设备损坏。新安装的空调制冷差，可能是抽真空不彻底导致系统内混有空气，严重影响冷效。

空调使用三五年后制冷下降，不一定是缺氟，不要被无良维修工忽悠，先清洗内外机。日常维护不仅能延长空调使用寿命，还能保持制冷效率和室内空气健康。

夏季或冬季高频使用期间，建议每隔两周至一个月清洗一次过滤网。

Your message



示例 2: 故障排除

用户输入：

如何清理空调？

系统输出：



如何清理空调？



如何清理空调？ 定期启动自清洁功能（建议每1-2个月一次），可以有效减少人工深度拆洗的频次。

对于安装在厨房的专用空调，需每月取下金属防油滤网进行热水浸泡去油清洗。建议每年换季首次开机前，对蒸发器进行一次深度免拆清洗。

购买正规品牌的空调清洗剂（喷雾型），摇匀后均匀喷洒在蒸发器翅片上。

喷洒完毕后静置15-20分钟，让清洁泡沫充分溶解翅片深处的污垢。长期使用的空调遥控器按键缝隙中容易藏污纳垢，可用酒精棉签清理。

内机导风板的轴承转轴处如果发出“吱吱”声，可涂抹极微量的凡士林润滑，切忌用液态机油以免滴落。清理空调可以按照以下步骤进行，既能保持空调性能，又能延长使用寿命：

1. 定期使用自清洁功能

- 建议每1-2个月启动一次空调的自清洁功能（如果机型支持），可有效减少灰尘和霉菌堆积，降低人工深度清洗频率。

2. 清洗滤网

Your message



示例 3: 维护保养

用户输入：

推荐空调

系统输出：

 推荐空调

 推荐空调 选择内外机均使用直流无刷电机的“全直流变频”空调，性能最佳。

劣质空调常用交流电机替代直流电机以降低成本，耗电且噪音大。

确认遥控器是否带背光显示，方便夜间不开灯操作温度调节。选购空调首看匹数，面积与匹数必须匹配，买大不买小。

10-12平米小卧室建议选择1匹空调。

12-20平米标准卧室强烈建议选择1.5匹空调。

20-30平米小客厅适合2匹壁挂机或柜机。警惕“假一级能效”，需通过正规渠道核实能效备案号。

网购空调认准官方旗舰店或自营渠道，避免买到翻新机或库存机。

线下实体店购买时，注意比对线上同款型号，部分型号为线下特供。选购空调时，建议重点关注以下几个方面，以确保性能、节能与使用体验：

Your message 

九、系统特色与创新点

9.1 架构创新

💡 LLM 主导设计

- LLM（大模型）是决策者，RAG 是工具
- 不是 RAG 主导的被动回答
- 真正实现“工具调用”而非“流水线处理”

💡 纯文本检索结果

- RAG 返回的是原始资料，不带格式标签
- LLM 进行自主整合和组织
- 避免生成结果“硬生生”地拼接参考资料

9.2 性能优化

⚡ 递归分块策略

- 按段落→句子→空格→字符递归分割
- 保持语义完整性

- 减少语境丢失

⚡ 向量化优化

- 使用 DashScopeEmbeddings 高质量嵌入
- 维度 1024, 精度高
- 支持中英文混合

9.3 用户体验

🎯 流式输出

- 实时显示生成结果
- 提升用户体验

🔧 多工具协作

- RAG 用于知识检索
- 其他工具扩展功能（可选）

十、运行指南

快速启动

```
# 1. 激活 Conda 环境

# 2. 安装依赖
pip install -r requirements.txt
# 3. 输入阿里apikey
/.env.example
/config/key.yml
# 3. 加载并清洗数据（首次运行）
python data_loader_cleaner.py

# 4. 启动应用
streamlit run app.py

# 5. 打开浏览器访问
# http://localhost:8501
```

数据更新

```
# 新增知识库文件到 data/ 文件夹
# 然后重新运行：
python data_loader_cleaner.py
```

十一、项目成果总结

功能完整性

-  数据加载与清洗模块
-  文本分块处理
-  向量索引构建
-  相似度检索
-  LLM 生成问答
-  完整 ReAct 代理框架

应用场景

-  选购咨询
-  使用说明
-  故障排除
-  维护保养
-  能耗建议

数据质量

-  500+ 条故障排除方案
-  450+ 个向量块
-  自动去重和验证
-  分块均衡 (平均 189 字符/块)

检索效果

-  高相关性 (Top-3 检索)
-  上下文完整性 (20 字符重叠)
-  多个数据源融合

十二、参考资源

- LangChain: <https://python.langchain.com>
- LangGraph: <https://langchain-ai.github.io/langgraph>
- Chroma: <https://docs.trychroma.com>
- Streamlit: <https://docs.streamlit.io>
- 阿里通义千问: <https://dashscope.aliyun.com>